

# Authentication Module Specification

## REAL LIFE SOLUTION FOR WASTE MANAGEMENT SYSTEM

|                   |                                             |
|-------------------|---------------------------------------------|
| Project Scope:    | Academic Mini-Project Documentation         |
| Module Name:      | User Authentication & Access Control System |
| Document Version: | v1.0                                        |

### 1. Introduction & Objective

The Authentication Module serves as the security gateway for the **Smart Waste Management System**. In any real-world civic infrastructure development, tracking data integrity and defining clear organizational responsibilities is essential. This module ensures that only authenticated individuals gain entry into the management interface and strictly partitions feature sets based on formalized organizational duties.

### 2. Role-Based Access Control (RBAC) Matrix

To preserve operations integrity and ensure a clean workflow, the module handles three specific core roles within the application ecology:

| User Role            | Access Privilege  | Functional Context                                                                                                                                                        |
|----------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| System Administrator | Global Controls   | Authorized to monitor city-wide analytics dashboards, reconfigure collection routing limits, add or terminate collection driver profiles, and oversee system metrics.     |
| Collection Driver    | Operational Views | Restricted to modern mapping components. Authorized to update individual bin fill tallies, signal completion tasks ("Emptied"), and file structural damage notifications. |
| Citizen / Area User  | Public Interfaces | Authorized to flag localized overflows, review local municipal service schedules, and track individual recycling contribution benchmarks.                                 |

### 3. Operational Authentication Flow

The logic chain executes sequentially from interface entry point to system dashboard distribution:

- Credential Dispatch:** The user accesses the system portal and submits verification credentials (Email ID / Password).

2. **Client-Side Validation:** The interface confirms fields comply with formal rules (e.g., non-empty strings, structural email configuration) before transmission.
3. **Secure Cryptographic Evaluation:** The system backend checks the database records for the unique email, passes the password string to an explicit hashing algorithm, and validates it against the stored hash safely.
4. **Token Issuance:** Upon successful match confirmation, the server generates a JSON Web Token (JWT) or secure session file mapping the user's explicit role flags.
5. **Dynamic Interface Routing:** The client engine evaluates the returned metadata token and automatically delivers the precise layout (Admin Console, Driver App, or Citizen Portal).

## 4. Core Schema Structure

The standard user profile model structure for the relational/non-relational mini-project database layer maps as follows:

| Field Name           | Data Format Type | Database Rule     | Functional Use Case                                                        |
|----------------------|------------------|-------------------|----------------------------------------------------------------------------|
| <b>user_id</b>       | VARCHAR / UUID   | Primary Key       | A unique identifier assigned to every user account dynamically.            |
| <b>full_name</b>     | VARCHAR(100)     | NOT NULL          | The documented name of the account operator.                               |
| <b>email_address</b> | VARCHAR(150)     | UNIQUE / NOT NULL | The unique string serving as the system account username login index.      |
| <b>password_hash</b> | VARCHAR(255)     | NOT NULL          | The salted and encrypted product of the text password string.              |
| <b>user_role</b>     | ENUM / STRING    | CHECK CONSTRAINT  | Restricted exclusively to values: 'admin', 'driver', or 'citizen'.         |
| <b>created_at</b>    | TIMESTAMP        | DEFAULT CURRENT   | A system timestamp documenting the initialization sequence of the account. |

## 5. Security Standards for Implementation

**Important Academic Best Practice:** Even within a localized mini-project framework, security standards add massive valuation to evaluation metrics.

- **Password Hashing:** Avoid storing plain text information. Implement standardized industry hashing engines like bcrypt or PBKDF2 inside the chosen language code.

- **Input Sanitization:** Escape and filter all incoming argument variables to actively defend the operational system pipeline from SQL Injection vectors.